22AIE303 – DATABASE MANAGEMENT SYSTEMS

HOTEL MANAGEMENT SYSTEM

PROJECT REPORT

Group Number: 3

TEAM MEMBERS

CB.SC.U4AIE24116 – J. Lakshmi Tejaswi
CB.SC.U4AIE24130 – Vishnu Teja
CB.SC.U4AIE24135 – Mithul Akshay
CB.SC.U4AIE24136 – Mohan Innani

1. Project Title

Hotel Management System — a desktop database application for managing hotel room types, rooms, guests, reservations, check-in/check-out status, payments, and operational reports.

2. Team Members

The project team consists of the following members:

Role / Field	Details
Project title	Hotel Management System
Course / code	22AIE303 – Database Management Systems
Team member(s)	J. Lakshmi Tejaswi; Vishnu Teja; Mithul Akshay; Mohan Innani
Roll number(s)	CB.SC.U4AIE24116; CB.SC.U4AIE24130; CB.SC.U4AIE24135; CB.SC.U4AIE24136
Batch / Group	Group 3
Faculty / Institution	[Enter if required]
GitHub repository URL	Not provided in the uploaded files.

3. Introduction and Problem Statement

A hotel handles interrelated information about room categories, physical rooms, guests, bookings, and payments. Maintaining these records separately or manually can lead to duplicate guest records, inconsistent room tariffs, overlapping reservations, incorrect room status, and difficulty generating revenue or occupancy summaries.

The Hotel Management System addresses these problems through a relational MySQL database and a Python Tkinter desktop interface. Staff authenticate through role-labelled accounts and use the application to create, view, search, update, and delete records; manage reservations; check guests in and out; record payments; and generate reports.

Objectives

- Centralise hotel operational data in a relational database.
- Maintain referential integrity using primary keys, foreign keys, UNIQUE constraints, CHECK constraints, and controlled delete rules.
- Prevent overlapping room reservations through date-overlap checks in the application.
- Calculate stay duration and room charges from check-in/check-out dates and the room-type tariff.
- Automate room-status recalculation using a stored procedure and triggers.
- Provide summary reports using joins, aggregate functions, GROUP BY, and HAVING.

Scope and technology

Component	Implementation
Frontend	Python desktop GUI using Tkinter
Database connector	mysql-connector-python
DBMS	MySQL 8.0; InnoDB; utf8mb4
Core modules	Login, dashboard, room types, rooms, guests, bookings, payments, reports
SQL assets	DDL, constraints, indexes, views, stored procedures, triggers, sample data
Repository	No GitHub URL was supplied in the uploaded files

4. Conceptual and Logical Database Design

The design separates data into six core entities. Room tariff and capacity are stored once per room type rather than repeated on every room. Guest identity details are stored in GUESTS; each booking refers to one guest and one room; each payment refers to a booking. The USERS table stores staff login credentials and role labels.

Entity	Purpose	Primary key	Key relationships
USERS	Staff authentication and role	user_id	Independent login entity
ROOM_TYPES	Room category, capacity and nightly rate	room_type_id	One room type can classify many rooms
ROOMS	Physical room number, floor and status	room_id	Many rooms belong to one room type
GUESTS	Guest identity and contact details	guest_id	One guest can have many bookings
BOOKINGS	Reservation dates, party size and status	booking_id	Each booking references one guest and one room
PAYMENTS	Amount, method, date and payment status	payment_id	Each payment references one booking; multiple payments per booking are structurally allowed

Relationship cardinalities

- ROOM_TYPES 1 : N ROOMS
- GUESTS 1 : N BOOKINGS
- ROOMS 1 : N BOOKINGS over time
- BOOKINGS 1 : N PAYMENTS (the schema permits multiple payments per booking)

Conceptual rules: check-out must be later than check-in; guest count must be between 1 and 10; room capacity must be between 1 and 10; room price must be positive; payment amount cannot be negative. Room status is maintained as Available, Occupied, or Maintenance. A future booking does not by itself mark a room Occupied.

System architecture

The Tkinter GUI calls functions in database.py. That module uses mysql-connector-python to connect to MySQL using db_config.py. The SQL schema and business-side database logic reside in MySQL. SQL statements are parameterised in the backend. The uploaded code uses a single configured database connection per operation rather than demonstrating a connection pool.

5. Normalization and Functional Dependencies

The schema is designed to approximately Third Normal Form (3NF): each table represents one entity or transaction, repeating groups are avoided, and non-key attributes are stored with the key that determines them. The following functional dependencies follow from the declared keys and uniqueness constraints.

Relation	Representative functional dependencies	Normalization rationale
USERS	user_id → username, password, role, created_at; username → user_id, password, role, created_at	Username is UNIQUE, so login identity is not duplicated.
ROOM_TYPES	room_type_id → room_type_name, capacity, price_per_night; room_type_name → room_type_id, capacity, price_per_night	Category tariff and capacity are stored once per category.
ROOMS	room_id → room_number, room_type_id, floor, status; room_number → room_id, room_type_id, floor, status	Room identity is independent of the category description.
GUESTS	guest_id → guest details; phone → guest_id and guest details; (id_proof, id_number) → guest_id and guest details	Surrogate primary key plus natural uniqueness constraints prevent duplicate contact/identity entries.
BOOKINGS	booking_id → guest_id, room_id, dates, number_of_guests, booking_date, booking_status	Reservation attributes depend on the booking identifier.

PAYMENTS	payment_id → booking_id, amount, payment_method, payment_date, payment_status	Payment transaction details are separated from booking and guest details.

1NF

Attributes contain single values rather than repeating groups. For example, a booking row contains one guest reference and one room reference.

2NF

All core tables use a single-column surrogate primary key, so there are no partial dependencies on a subset of a composite primary key. The composite UNIQUE constraint on (id_proof, id_number) in GUESTS enforces identity-document uniqueness but is not the table's primary key.

3NF

Room-type attributes such as capacity and price_per_night are held in ROOM_TYPES, not repeated in ROOMS. This avoids the transitive dependency that would arise if a room row stored a room-type name and then the room-type name determined the tariff. Booking and payment facts are separated into their own relations.

Controlled redundancy: PAYMENTS.amount is stored as a transaction value, while the bill view/procedure calculates a room charge from stay duration and the current room tariff. The stored payment amount can represent the amount actually charged/paid at the time, whereas the calculated room charge can change if a tariff is edited. The application does not currently wrap booking and payment operations in one explicit transaction, according to the setup guide.

6. Advanced SQL Implementation and Query Design

The schema includes integrity constraints, indexes, views, stored procedures, and triggers. The application backend also uses joins, a date-overlap subquery, parameterised statements, and aggregate reporting.

6.1 DDL constraints

- PRIMARY KEY: auto-increment identifiers in all six tables.
- FOREIGN KEY: rooms → room_types; bookings → guests and rooms; payments → bookings.
- UNIQUE: username, room_type_name, room_number, guest phone, and (id_proof, id_number).
- CHECK: room capacity, positive room tariff, floor range, valid booking dates, guest count, and non-negative payment amount.
- ON DELETE: guest deletion cascades to bookings and payment rows; room-type deletion is RESTRICTed when rooms reference it; room deletion is RESTRICTed when bookings reference it. A guest-delete trigger explicitly deletes bookings so booking-delete triggers can refresh room status.

6.2 Advanced queries and purpose

Technique	Implementation / purpose
Multi-table JOIN	Booking detail query joins BOOKINGS, GUESTS, ROOMS and ROOM_TYPES to display guest, room, category, dates, nights and calculated charge.
Subquery / NOT EXISTS logic	Available-room search excludes rooms whose bookings overlap the requested period and excludes rooms marked Maintenance.
Date overlap predicate	An existing booking conflicts when existing.check_in_date < requested_check_out AND existing.check_out_date > requested_check_in_date. This correctly treats a check-out date as available for a new same-day check-in.
Aggregate functions	Dashboard and reports use COUNT and SUM to calculate room counts, active bookings, paid revenue and pending payments.
GROUP BY	Room availability report groups room counts by room type; revenue report groups transaction count and amount by payment method.

HAVING	Top-guest report groups bookings per guest and filters groups using HAVING COUNT(booking_id) >= 1.
Views	vw_room_availability, vw_booking_details and vw_payment_summary package frequently used joins for consistent reporting.
Stored procedures	sp_generate_bill returns a booking's guest, room, tariff, number of nights and calculated room amount. sp_refresh_room_status recalculates room state.
Triggers	Booking insert/update/delete triggers call the room-status procedure. A guest BEFORE DELETE trigger removes bookings explicitly to ensure booking-delete triggers run.
Indexes	Indexes cover booking date ranges, booking status, guest names, and room status to support common filtering and lookup operations.

6.3 Representative SQL

A. Booking detail with joins

```
SELECT b.booking_id, CONCAT(g.first_name,' ',IFNULL(g.last_name,'')) AS guest_name,
      r.room_number, rt.room_type_name,
      DATEDIFF(b.check_out_date,b.check_in_date) AS nights,
      DATEDIFF(b.check_out_date,b.check_in_date) * rt.price_per_night AS total_amount,
      b.booking_status
FROM bookings b
JOIN guests g ON b.guest_id = g.guest_id
JOIN rooms r ON b.room_id = r.room_id
JOIN room_types rt ON r.room_type_id = rt.room_type_id;
```

B. Revenue by payment method

```
SELECT payment_method, COUNT(*) AS txn_count, SUM(amount) AS total
FROM payments
GROUP BY payment_method
ORDER BY total DESC;
```

C. Find rooms with no overlapping reservation

```
SELECT r.room_id, r.room_number, rt.room_type_name, rt.price_per_night
FROM rooms r JOIN room_types rt ON r.room_type_id = rt.room_type_id
WHERE r.status <> 'Maintenance'
 AND NOT EXISTS (
  SELECT 1 FROM bookings b
  WHERE b.room_id = r.room_id
   AND b.booking_status IN ('Booked','Checked-In')
   AND b.check_in_date < %s
   AND b.check_out_date > %s
 );
```

D. Generate bill using stored procedure

```
CALL sp_generate_bill(%s);
```

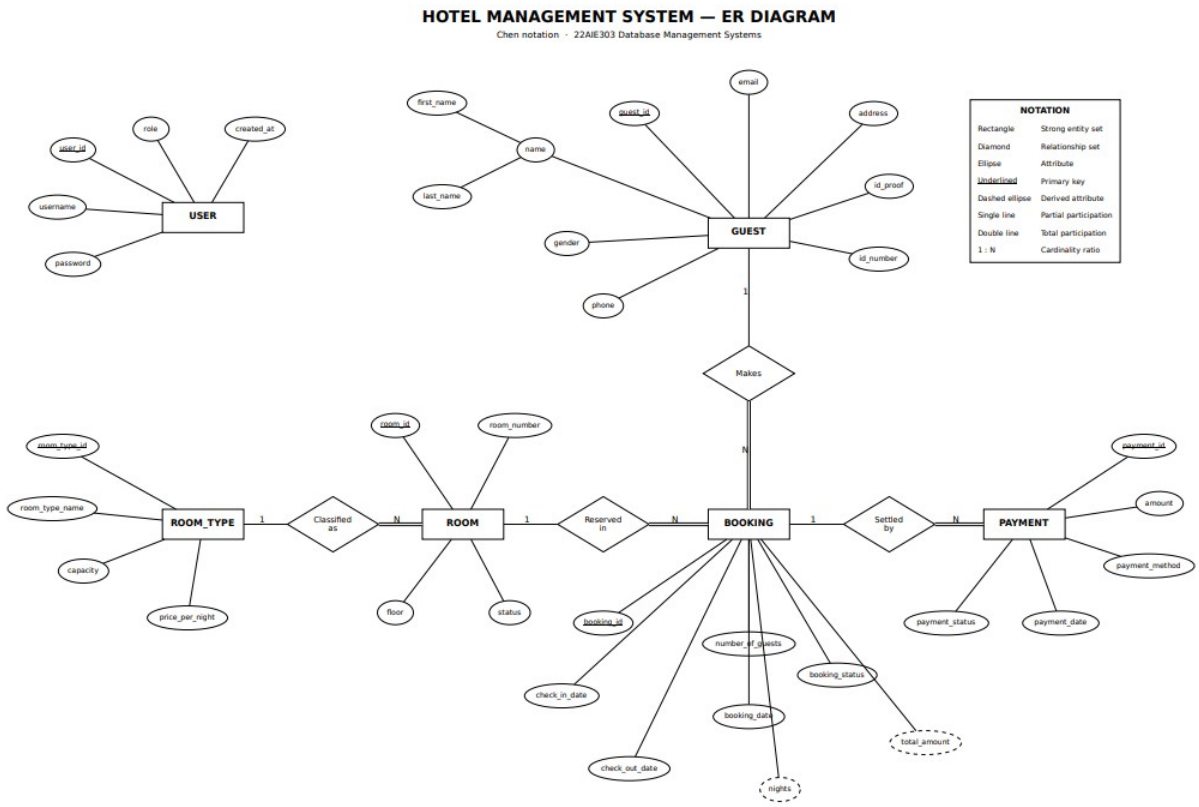
6.4 Security and limitations

The backend uses parameter placeholders for values, reducing SQL-injection risk for parameterised operations. Passwords are hashed using SHA-256, as indicated by the code and setup guide; SHA-256 alone is not a password-specific slow hash and should be replaced with a salted password-hashing scheme such as Argon2id or bcrypt for production. The provided db_config.py contains a local database password; it should not be committed to a public repository and should be moved to environment variables or a secrets manager. The date-overlap check is performed in application logic, so concurrent requests could still race; a transaction/locking strategy or database-level enforcement would strengthen this rule.

7. ER Diagram and UML Diagram

The ER diagram below models the principal entities, their attributes, primary keys, and relationships in the Hotel Management System.

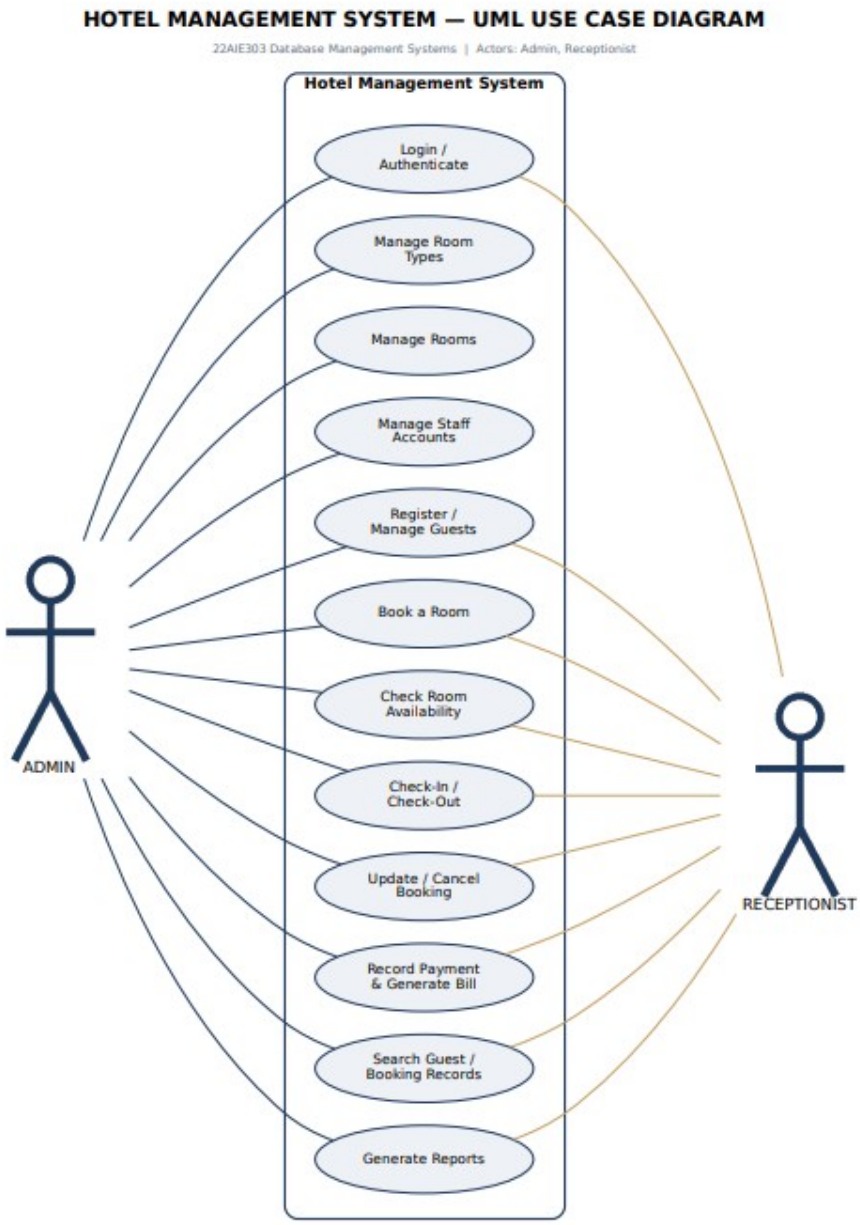
ER Diagram



ER model summary: ROOM_TYPE classifies ROOM; GUEST makes BOOKINGS; ROOM is RESERVED in BOOKINGS; BOOKINGS are SETTLED by PAYMENT. USERS supports staff authentication.

The UML use-case diagram below identifies Admin and Receptionist actors and the principal functions available in the Hotel Management System.

UML Use-Case Diagram



8. Relational Schema

- USERS(user_id PK, username UNIQUE, password, role, created_at)
- ROOM_TYPES(room_type_id PK, room_type_name UNIQUE, capacity, price_per_night)
- ROOMS(room_id PK, room_number UNIQUE, room_type_id FK → ROOM_TYPES.room_type_id, floor, status)
- GUESTS(guest_id PK, first_name, last_name, gender, phone UNIQUE, email, address, id_proof, id_number, UNIQUE(id_proof, id_number))
- BOOKINGS(booking_id PK, guest_id FK → GUESTS.guest_id, room_id FK → ROOMS.room_id, check_in_date, check_out_date, number_of_guests, booking_date, booking_status)
- PAYMENTS(payment_id PK, booking_id FK → BOOKINGS.booking_id, amount, payment_method, payment_date, payment_status)

9. Database Tables — Structures, Attributes, Keys and Sample Records

USERS

Attribute	Data type / definition	Key / constraint
user_id	INT AUTO_INCREMENT	PK
username	VARCHAR(50) NOT NULL	UNIQUE
password	VARCHAR(64) NOT NULL	SHA-256 digest in sample
role	ENUM(Admin, Receptionist)	DEFAULT Receptionist
created_at	TIMESTAMP	DEFAULT current timestamp

user_id	username	password	role	created_at
1	admin	[SHA-256 digest]	Admin	auto
2	reception	[SHA-256 digest]	Receptionist	auto

ROOM_TYPES

Attribute	Data type / definition	Key / constraint
room_type_id	INT AUTO_INCREMENT	PK
room_type_name	VARCHAR(40) NOT NULL	UNIQUE
capacity	INT NOT NULL	CHECK 1–10
price_per_night	DECIMAL(10,2) NOT NULL	CHECK > 0

id	room_type_name	capacity	price/night
1	Standard	2	1800.00
2	Deluxe	3	3200.00
3	Executive Suite	4	5500.00
4	Family Room	5	4200.00

ROOMS

Attribute	Data type / definition	Key / constraint
room_id	INT AUTO_INCREMENT	PK
room_number	VARCHAR(10) NOT NULL	UNIQUE
room_type_id	INT NOT NULL	FK → ROOM_TYPES
floor	INT NOT NULL	CHECK 0–50
status	ENUM(Available, Occupied, Maintenance)	DEFAULT Available

room_id	room_number	room_type_id	floor	status
1	101	1	1	Available*
2	102	1	1	Available
3	103	1	1	Maintenance
4	201	2	2	Available

GUESTS

Attribute	Data type / definition	Key / constraint
guest_id	INT AUTO_INCREMENT	PK
first_name	VARCHAR(50) NOT NULL	
last_name	VARCHAR(50)	NULL allowed
gender	ENUM(Male, Female, Other)	
phone	VARCHAR(15) NOT NULL	UNIQUE
email	VARCHAR(80)	
address	VARCHAR(150)	
id_proof	ENUM(Aadhaar, Passport, Driving License, Voter ID)	
id_number	VARCHAR(40) NOT NULL	UNIQUE with id_proof

guest_id	first_name	last_name	gender	phone	email	id_proof	id_number
1	Arjun	Menon	Male	9876543210	arjun.m@example.com	Aadhaar	4321-8765-1098
2	Priya	Raghavan	Female	9812345678	priya.r@example.com	Aadhaar	1122-3344-5566

BOOKINGS

Attribute	Data type / definition	Key / constraint
booking_id	INT AUTO_INCREMENT	PK
guest_id	INT NOT NULL	FK → GUESTS
room_id	INT NOT NULL	FK → ROOMS
check_in_date	DATE NOT NULL	CHECK out > in
check_out_date	DATE NOT NULL	
number_of_guests	INT NOT NULL DEFAULT 1	CHECK 1–10
booking_date	DATE NOT NULL	
booking_status	ENUM(Booked, Checked-In, Checked-Out, Cancelled)	DEFAULT Booked

booking_id	guest_id	room_id	check-in	check-out	guests	booking date	status
1	1	1	2026-08-12	2026-08-15	2	2026-08-10	Checked-In
2	2	4	2026-08-13	2026-08-16	3	2026-08-10	Booked

PAYMENTS

Attribute	Data type / definition	Key / constraint
payment_id	INT AUTO_INCREMENT	PK
booking_id	INT NOT NULL	FK → BOOKINGS
amount	DECIMAL(10,2) NOT NULL	CHECK >= 0
payment_method	ENUM(Cash, Card, UPI, Net Banking)	
payment_date	DATE NOT NULL	
payment_status	ENUM(Paid, Pending, Refunded)	DEFAULT Pending

payment_id	booking_id	amount	method	payment date	status
1	1	5400.00	UPI	2026-08-12	Paid
2	2	9600.00	Card	2026-08-13	Pending

*Room 101 is linked to a Checked-In sample booking. The booking triggers recalculate its room status as Occupied when the schema is created and sample data is inserted, subject to trigger execution.

10. Conclusion and Future Scope

The Hotel Management System demonstrates the use of a relational schema to support core hotel operations. Its design separates room categories, rooms, guests, reservations and payments, while enforcing integrity through keys and constraints. Views simplify repeated joins, stored procedures encapsulate billing and room-status logic, and triggers keep room status consistent with booking status. The Tkinter application provides CRUD workflows and reports backed by MySQL.

Future enhancements

- Move password storage from plain SHA-256 to salted Argon2id/bcrypt and remove hard-coded credentials from source control.

- Add transaction management and concurrency-safe reservation creation to prevent race conditions in simultaneous bookings.
- Add audit logging, role-based authorization checks, automated tests and backup/restore procedures.
- Add a database-level strategy for enforcing reservation overlap rules.
- Support bookings containing multiple rooms through a BOOKING_ROOMS junction table if required.
- Add tax, discounts, cancellation fees, invoice numbers, partial payments and refunds with clearly defined business rules.
- Deploy the database securely and provide a reproducible README with a verified GitHub repository URL.

Appendix A — Setup and Demonstration

- Install MySQL 8.0 and Python 3.
- Install the connector: pip install mysql-connector-python.
- Run the schema: mysql -u root -p < 01_schema.sql (the script drops and recreates hotel_db; existing data will be lost).
- Update db_config.py with local database credentials; do not expose real credentials publicly.
- Launch the application: python app.py.
- Demonstrate login, dashboard, guest CRUD/search, room and booking operations, overlapping booking rejection, check-in/check-out, payment entry, and reports.

Appendix B — Source files reviewed

- 01_schema.sql — database schema, constraints, indexes, views, stored procedures, triggers, sample data.
- app.py — Tkinter login and application interface.
- database.py — database connection and query functions.
- db_config.py — local MySQL connection configuration.
- ER_Chen.pdf — ER diagram supplied in ZIP.
- Use_Case_Diagram.pdf — UML/use-case diagram supplied in ZIP.
- SETUP_AND_DEMO_GUIDE.md — setup, demonstration and viva guidance.

Submission note: The GitHub repository URL was not present in the uploaded files. Add the actual repository URL after publishing the project.